

Relatório Técnico: Compressão e Descompressão Paralela de Arquivos

1. Identificação da Instituição, Curso e Disciplina/Professor

Instituição: [preencher]

Curso: [preencher]

Disciplina: [preencher]

Professor(a): [preencher]

Ambiente de execução: Linux

Linguagem: C padrão, com uso de pthread e sem orientação a objetos.

2. Identificação dos Participantes

Participante 1: [preencher nome]

Participante 2: [preencher nome]

Participante 3: [preencher nome, se houver]

Observação: os nomes e dados institucionais devem ser preenchidos pelo grupo antes da entrega final.

3. Tema da Paralelização

O tema escolhido foi a compressão e descompressão de arquivos usando o algoritmo LZ77, com paralelização por blocos independentes. O projeto foi baseado no repositório sequencial [cstdvd/lz77](https://github.com/cstdvd/lz77), que implementa a lógica básica do algoritmo LZ77, incluindo codificação em bits, árvore binária para busca de repetições e rotina de descompressão.

O problema resolvido é reduzir o tamanho de arquivos por meio de compressão sem perdas e, posteriormente, recuperar exatamente o conteúdo original. A versão paralela busca acelerar principalmente as etapas de compressão e descompressão dividindo o arquivo em blocos processados por múltiplas threads.

4. Explicação da Aplicação Sequencial

A aplicação sequencial implementa o algoritmo LZ77. O algoritmo percorre o arquivo original mantendo uma janela deslizante formada por duas partes:

? 'search-buffer': região com dados já processados, usada como dicionário.

? 'lookahead': região com os próximos bytes que ainda serão comprimidos.

Para cada posição, o compressor procura a maior sequência no lookahead que já apareceu no search-buffer. O resultado é um token composto por:

? 'off': distância para trás onde a sequência repetida começa.

? 'len': quantidade de bytes repetidos.

? 'next': próximo byte literal após a repetição.

Na implementação sequencial, a função `encode_with_timing()` em `lz77.c` executa a compressão. Ela usa uma árvore binária, implementada em `tree.c`, para acelerar a busca por sequências repetidas. A função `writetoken()` grava cada token em formato compacto no arquivo de saída usando a biblioteca de escrita bit a bit de `bitio.c`.

A descompressão sequencial é feita por `decode_with_timing()`. Ela lê os tokens com `readtoken()` e reconstrói o arquivo original copiando bytes anteriores do buffer de reconstrução ou escrevendo o byte literal `next`.

Principais arquivos da versão sequencial:

- ? 'lz77/lz77.c': compressão, descompressão, geração e leitura de tokens.
- ? 'lz77/lz77.h': definições de tokens, estruturas de tempo e protótipos.
- ? 'lz77/tree.c' e 'lz77/tree.h': Árvore binária usada na busca de padrões.
- ? 'lz77/bitio.c' e 'lz77/bitio.h': leitura e escrita em nível de bits.
- ? 'lz77/main.c': interface por linha de comando.

5. Explicação da Aplicação Paralela

A aplicação paralela foi construída a partir da versão sequencial, dividindo o arquivo de entrada em blocos independentes. Cada thread recebe um intervalo [start, end) do arquivo original e comprime apenas os bytes desse bloco.

Na compressão paralela, a função principal é `compress_parallel_file()`, localizada em `lz77/parallel/parallel.c`. O funcionamento geral é:

1. O arquivo de entrada é lido para um buffer em memória.
2. O tamanho do arquivo é dividido pelo número de threads.
3. Cada thread recebe um `ThreadData` com ponteiro para o buffer, início e fim do bloco, parâmetros LZ77 e caminho de arquivo temporário.
4. Cada thread executa `compress_thread()`, que chama `compress_block_to_file()`.
5. Cada bloco comprimido é gravado em arquivo temporário.
6. A thread principal escreve o cabeçalho paralelo e concatena os blocos comprimidos na ordem correta.

O cabeçalho paralelo armazena metadados necessários para a descompressão paralela:

- ? marcador de arquivo paralelo;
- ? assinatura de validação;
- ? tamanho do 'search-buffer';
- ? tamanho do 'lookahead';
- ? quantidade de blocos;
- ? quantidade de tokens por bloco.

Na descompressão paralela, a função `decompress_parallel_file()` lê esses metadados, calcula a posição em bits de cada bloco comprimido e cria threads para descomprimir blocos independentes. Cada thread executa `decompress_thread()`, que chama `decode_tokens_from_file()`. O resultado de cada bloco é gravado em arquivo temporário e, ao final, a thread principal junta os blocos descomprimidos na ordem original.

As funções de `pthread` usadas são apenas as permitidas:

- ? `'pthread_create'`
- ? `'pthread_join'`
- ? `'pthread_exit'`
- ? `'pthread_mutex_lock'`
- ? `'pthread_mutex_unlock'`
- ? `'sem_wait'`
- ? `'sem_post'`
- ? `'sem_trywait'`

O mutex é usado para proteger mensagens e estatísticas compartilhadas. O semáforo controla a entrada de threads nas fases paralelas, principalmente na descompressão, quando o arquivo

pode ter mais blocos que o número de threads simultâneas pedido pelo usuário.

Modos de Uso

Compilação:

```
cd lz77
make
```

Compressão sequencial:

```
./lz77 -c -i ../arquivos/originais/entrada.txt -o ../arquivos/comprimidos/saida_seq.lz
```

Compressão paralela:

```
./lz77 -c -p 8 -i ../arquivos/originais/entrada.txt -o ../arquivos/comprimidos/saida_par.lz
```

Descompressão sequencial:

```
./lz77 -d -i ../arquivos/comprimidos/saida_seq.lz -o ../arquivos/descomprimidos/saida_seq.txt
```

Descompressão paralela:

```
./lz77 -d -p 8 -i ../arquivos/comprimidos/saida_par.lz -o
../arquivos/descomprimidos/saida_par.txt
```

Parâmetros aceitos:

- ? '-c': executa compressão.
- ? '-d': executa descompressão.
- ? '-p <threads>': usa versão paralela com número máximo de threads.
- ? '-i <arquivo>': arquivo de entrada.
- ? '-o <arquivo>': arquivo de saída.
- ? '-l <valor>': tamanho do lookahead.
- ? '-s <valor>': tamanho do search-buffer.
- ? '-h': mostra ajuda.

Neste projeto, o tamanho do problema é definido principalmente pelo arquivo de entrada escolhido. Também é possível variar os parâmetros -l e -s, que alteram o tamanho da janela de compressão e influenciam o custo computacional do algoritmo.

Métricas de Tempo Impressas

Compressão sequencial:

- ? 'Leitura do arquivo': tempo gasto lendo bytes do arquivo original.
- ? 'Preparacao': tempo de abertura de arquivos e preparação inicial.
- ? 'Compressao sequencial': tempo gasto na lógica LZ77 sequencial, desconsiderando leitura e escrita medidas separadamente.
- ? 'Escrita': tempo gasto escrevendo tokens e fazendo flush do arquivo comprimido.
- ? 'Tempo total sequencial': tempo completo da operação.

Compressão paralela:

- ? 'Leitura do arquivo': tempo para carregar o arquivo original em memória.
- ? 'Preparacao': tempo para abrir saída, alocar estruturas, inicializar mutex e semáforo.
- ? 'Compressao nas threads': tempo de parede desde a criação das threads até o término de todas elas. Não é soma dos tempos individuais; representa a duração real da fase

paralela.

? 'Escrita/copia final': tempo para escrever o cabeçalho paralelo e copiar os blocos temporários para o arquivo final.

? 'Tempo total paralelo': tempo total medido dentro da função paralela.

Descompressão sequencial:

? 'Leitura/metadados': tempo gasto lendo cabeçalho, metadados e tokens.

? 'Preparação': tempo de abertura dos arquivos.

? 'Descompressão sequencial': tempo gasto reconstruindo os dados.

? 'Escrita/junção final': tempo gasto escrevendo o arquivo descomprimido.

? 'Tempo total sequencial': tempo completo da operação.

Descompressão paralela:

? 'Leitura/metadados': tempo gasto lendo cabeçalho paralelo e calculando offsets dos blocos.

? 'Preparação': tempo para alocar estruturas e inicializar sincronização.

? 'Descompressão nas threads': tempo de parede da fase paralela de descompressão.

? 'Escrita/junção final': tempo para juntar os blocos temporários em ordem.

? 'Tempo total paralelo': tempo completo da descompressão paralela.

6. Avaliação de Desempenho da Execução

Os testes foram executados em ambiente Linux usando o arquivo:

arquivos/originais/biblia-em-txt.txt

Tamanho do arquivo original: 4.055.830 bytes.

Resultados de Compressão

Configuração	Threads	Tempo total (s)	Processamento (s)	Escrita (s)	Tamanho comprimido (bytes)
Sequencial	1	0,839464	0,686406	0,140223	2.112.769
Paralela	2	1,335148	0,616279	0,716585	2.113.272
Paralela	4	1,028282	0,330753	0,694827	2.114.249
Paralela	8	0,968799	0,252306	0,708937	2.116.374

Resultados de Descompressão

Configuração	Threads	Tempo total (s)	Processamento (s)	Escrita (s)
Sequencial sobre arquivo sequencial	1	0,436900	0,106159	0,127982
Sequencial sobre arquivo paralelo	1	0,442092	0,109625	0,129229
Paralela sobre arquivo paralelo	2	0,154446	0,150204	0,003316
Paralela sobre arquivo paralelo	4	0,074065	0,068445	0,004695
Paralela sobre arquivo paralelo	8	0,063148	0,058228	0,003912

Análise dos Resultados

Na compressão, a fase Compressão nas threads melhorou conforme o número de threads aumentou. Com 8 threads, a fase de processamento caiu de 0,686406 s na versão sequencial para 0,252306 s na fase paralela. Porém, o tempo total da compressão paralela ainda ficou maior que o sequencial. O motivo principal foi a fase Escrita/copia final, que ficou em torno de 0,70 s. Essa fase é serial e copia os blocos temporários para o arquivo final bit a bit.

Na descompressão, a versão paralela apresentou ganho claro. A descompressão sequencial do arquivo paralelo levou 0,442092 s, enquanto a descompressão paralela com 8 threads levou 0,063148 s. Isso ocorreu porque os metadados do arquivo paralelo permitem que cada thread

localize diretamente o início de seu bloco e reconstrua parte do arquivo de forma independente.

Os tamanhos comprimidos paralelos ficaram ligeiramente maiores que o sequencial. Isso acontece porque cada bloco é comprimido independentemente; assim, repetições que cruzariam fronteiras entre blocos não são aproveitadas.

Corretude dos Resultados

A corretude foi verificada comparando o arquivo original com os arquivos descomprimidos usando cmp. Nos experimentos realizados, os seguintes comandos não apresentaram diferenças:

```
cmp ../arquivos/originais/biblia-em-txt.txt
../arquivos/descomprimidos/relatorio_biblia_seq_decode.txt
cmp ../arquivos/originais/biblia-em-txt.txt
../arquivos/descomprimidos/relatorio_biblia_p8_seqdecode.txt
cmp ../arquivos/originais/biblia-em-txt.txt
../arquivos/descomprimidos/relatorio_biblia_p8_decode_p2.txt
cmp ../arquivos/originais/biblia-em-txt.txt
../arquivos/descomprimidos/relatorio_biblia_p8_decode_p4.txt
cmp ../arquivos/originais/biblia-em-txt.txt
../arquivos/descomprimidos/relatorio_biblia_p8_decode_p8.txt
```

Assim, os arquivos descomprimidos pela versão sequencial e paralela foram iguais ao arquivo original nos testes executados.

7. Atividades dos Participantes

Participante 1: [preencher nome]

Responsabilidade sugerida: estudo da versão sequencial, integração de main.c, testes de compressão sequencial e documentação do uso do programa.

Participante 2: [preencher nome]

Responsabilidade sugerida: implementação da compressão paralela em parallel/parallel.c, criação de threads, divisão dos blocos e sincronização com mutex/semaforo.

Participante 3: [preencher nome, se houver]

Responsabilidade sugerida: implementação da descompressão paralela, cabeçalho com metadados, verificação de corretude, coleta de tempos e análise de desempenho.

Observação: esta divisão deve ser ajustada para refletir fielmente o que cada integrante realizou.

8. Autoavaliação

O projeto conseguiu implementar compressão e descompressão LZ77 em versões sequencial e paralela. A aplicação permite definir arquivo de entrada, arquivo de saída, número de threads, tamanho do lookahead e tamanho do search-buffer.

As principais dificuldades foram:

- ? entender o formato original de escrita bit a bit;
- ? garantir que blocos comprimidos independentemente continuassem descomprimíveis;
- ? permitir descompressão paralela, o que exigiu armazenar metadados de blocos;
- ? medir separadamente leitura, processamento e escrita;
- ? controlar o custo da fase serial de escrita/cópia final.

O que funciona:

- ? compressão sequencial;

- ? descompressão sequencial;
- ? compressão paralela por blocos;
- ? descompressão paralela de arquivos gerados pela compressão paralela nova;
- ? medição detalhada de tempos;
- ? verificação experimental por comparação byte a byte com 'cmp'.

Limitações:

- ? a compressão paralela pode ser mais lenta que a sequencial em arquivos médios, pois a fase de escrita/cópia final ainda é serial e custosa;
- ? arquivos paralelos antigos, gerados antes da inclusão do cabeçalho com metadados, não podem ser descomprimidos em paralelo;
- ? a compressão por blocos independentes reduz a capacidade de encontrar repetições entre blocos;
- ? a verificação de corretude foi executada nos experimentos por comparação externa com 'cmp'; uma opção interna dedicada de verificação pode ser adicionada como melhoria futura.

Conclusão: a paralelização foi mais efetiva na descompressão do que na compressão. Na compressão, a parte computacional acelerou, mas o ganho foi reduzido pela escrita final. Na descompressão, o processamento por blocos independentes obteve redução significativa no tempo total.

Referência

Repositório base da implementação sequencial:

<https://github.com/cstdvd/lz77>

Grafico 1 - Tempo total de compressao

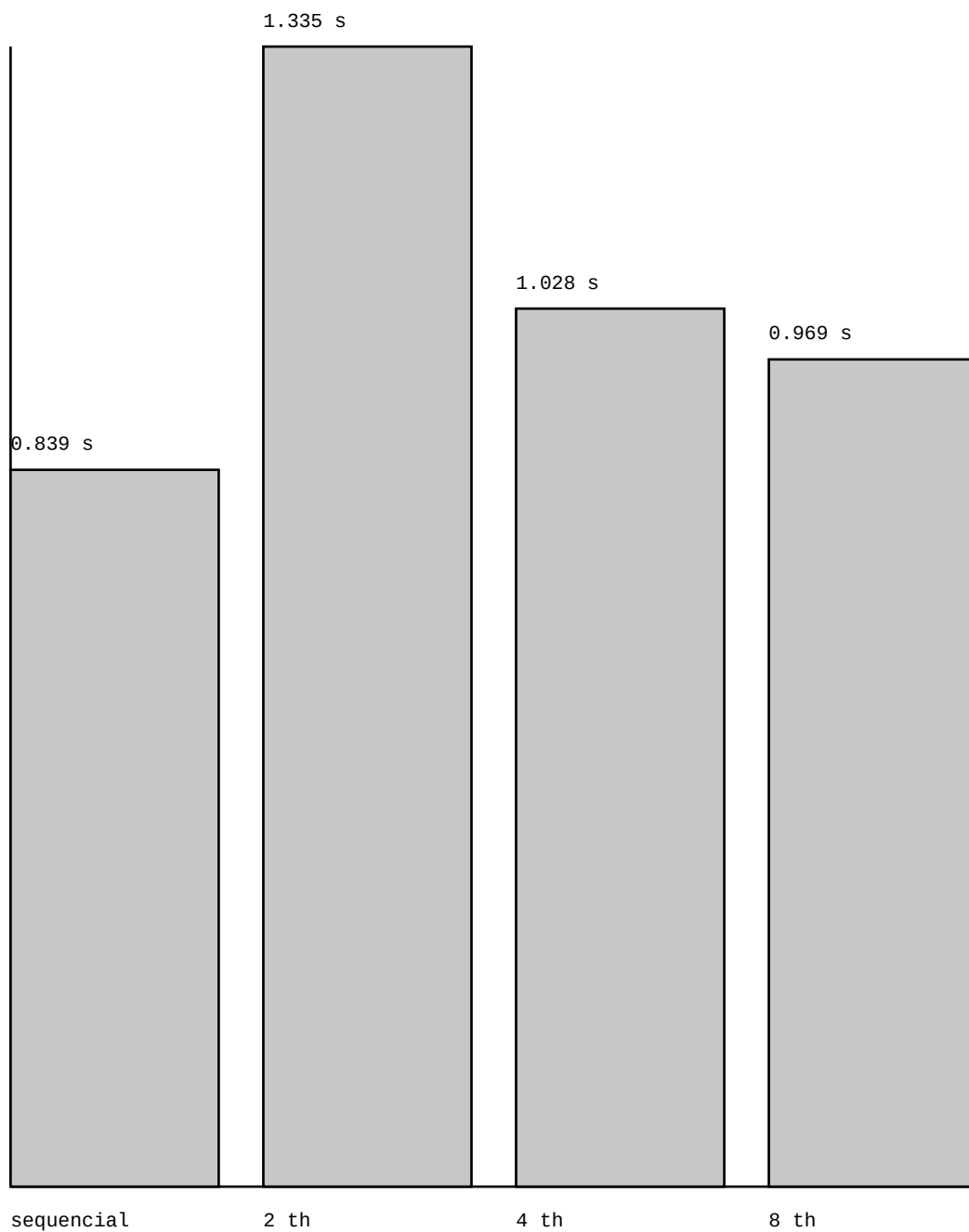


Grafico de barras gerado a partir dos tempos medidos no ambiente Linux do projeto.

Grafico 2 - Tempo de processamento da compressao

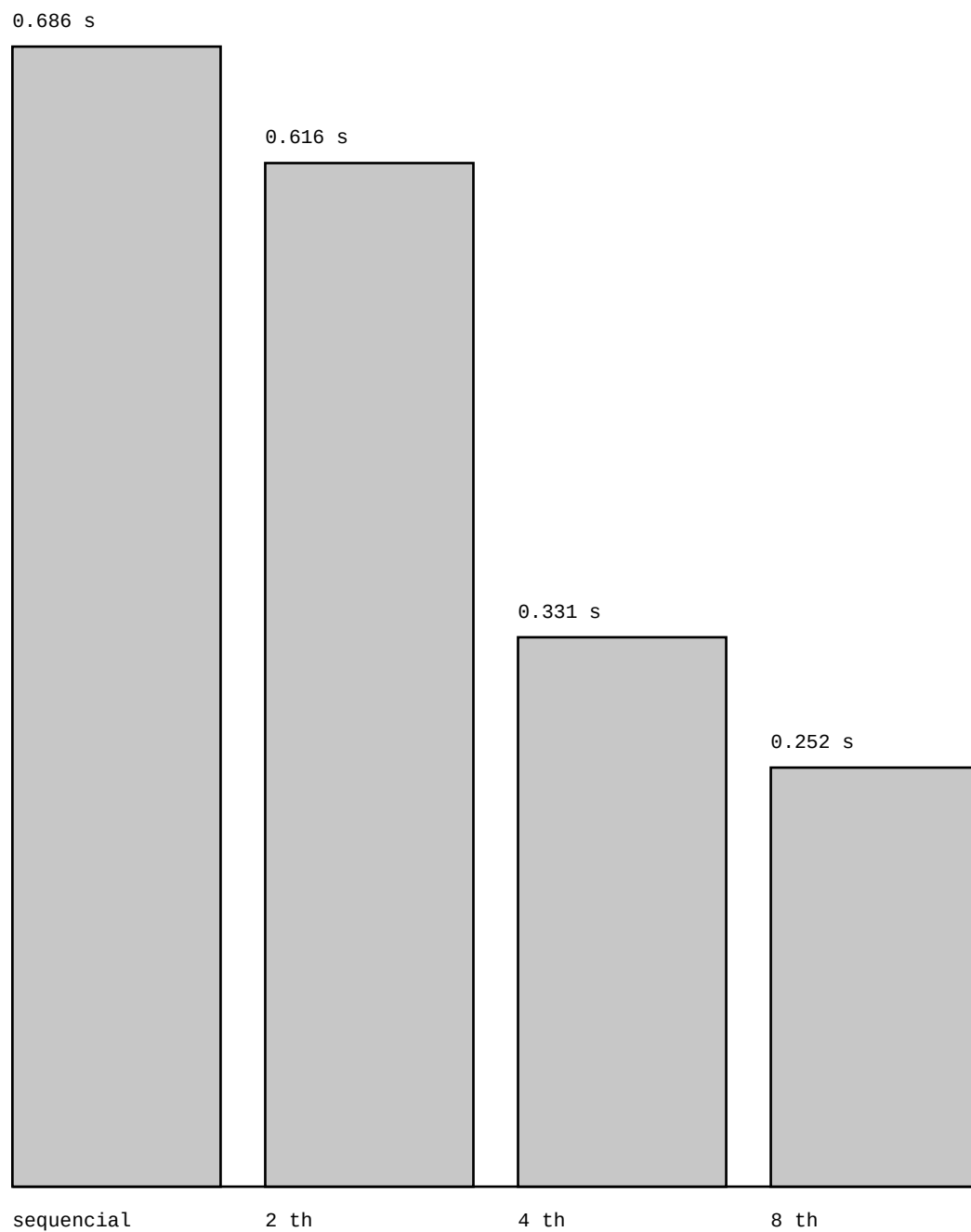


Grafico de barras gerado a partir dos tempos medidos no ambiente Linux do projeto.

Grafico 3 - Tempo total de descompressao

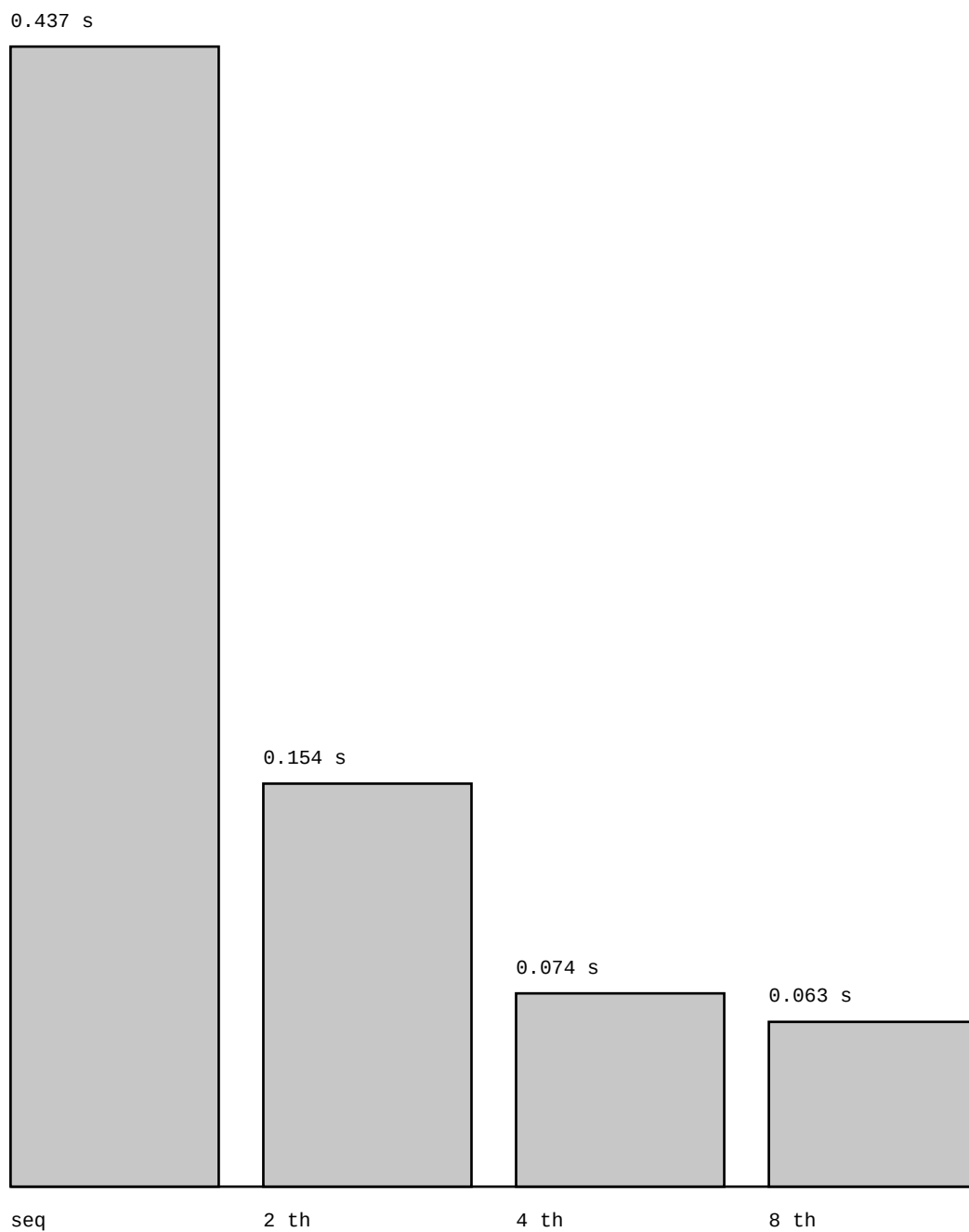


Grafico de barras gerado a partir dos tempos medidos no ambiente Linux do projeto.

Grafico 4 - Tempo de processamento da descompressao

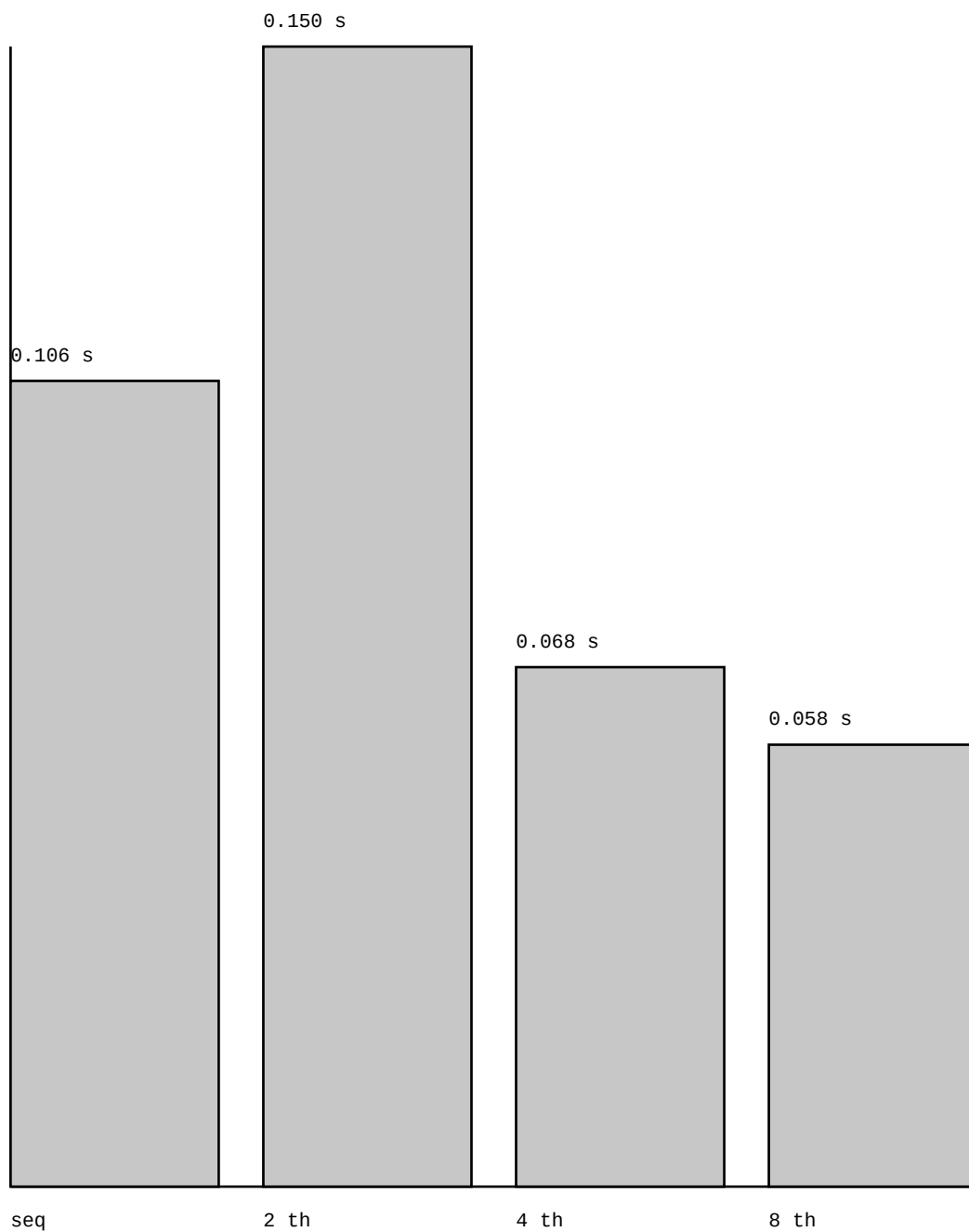


Grafico de barras gerado a partir dos tempos medidos no ambiente Linux do projeto.